

SeHAT – Smart e-Health Assistance Technology

SeHAT SmartCare AI – An Intelligent Mobile Health Assistant
using AI, RAG and Medical Language Models

A project report submitted in partial fulfillment for the award of the degree of

Bachelor of Technology

in

Computer Science & Engineering

Submitted by

Shivam Tiwari (2202840100205)

Abhishek Chauhan (2202840100010)

Raghvendra Bind (2202840100162)

Raj Kumar (2202840100164)

Under the guidance of

Dr. Anubhav Kumar Prasad

(Associate Professor)



DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING

UNITED INSTITUTE OF TECHNOLOGY

PRAYAGRAJ, UTTAR PRADESH 211010, INDIA.

(Affiliated to Dr. A.P.J. Abdul Kalam Technical University, Lucknow)

Declaration of Academic Ethics

We, the undersigned, students of Bachelor of Technology in Computer Science & Engineering, hereby declare that the work, titled "SeHAT – Smart e-Health Assistance Technology (SeHAT SmartCare AI)", presented in this report is original and is the result of our own investigations.

All external contributions, whether through literature, data, or media, have been explicitly acknowledged and referenced in accordance with standard academic practices. We certify that this document has been prepared in full compliance with the Institute's code of academic ethics. We have not engaged in any form of data fabrication, falsification, or plagiarism.

We acknowledge that any breach of this declaration shall render us liable to disciplinary proceedings by the Institute and may further invite legal or penal consequences from the original authors or copyright holders of utilized materials.

Date: 19 December 2025	Name of Student (Roll Number) Shivam Tiwari (2202840100205) Abhishek Chauhan (2202840100010) Raghvendra Bind (2202840100162) Raj Kumar (2202840100164)	Sign
------------------------	--	------

Certificate from the Project Guide

This is to certify that the project report titled "SeHAT – Smart e-Health Assistance Technology (SeHAT SmartCare AI – An Intelligent Mobile Health Assistant using AI, RAG and Medical Language Models)" is a bona fide record of the work carried out by:

- Shivam Tiwari (2202840100205)
- Abhishek Chauhan (2202840100010)
- Raghvendra Bind (2202840100162)
- Raj Kumar (2202840100164)

The work was carried out under my supervision and guidance, in partial fulfillment of the requirements for the award of the degree of Bachelor of Technology in Computer Science & Engineering during the academic year 2025–2026.

To the best of my knowledge,

1. the matter embodied in this project report has not been submitted to any other University or Institute for the award of any degree or diploma.
2. Has duly been completed.
3. Fulfills the requirement of the Ordinance relating to the Bachelor of Technology degree of the University.
4. Is up to the desired standard both in respect of contents and language for being referred to the examiners.

Date: 19 December 2025	Internal Supervisor	Head of Department
	Dr. Anubhav Kumar Prasad (Associate Professor)	_ [HOD Name] (Professor & Head)

Certificate from External Examiner

This is to certify that the Project Report titled "SeHAT – Smart e-Health Assistance Technology" submitted by:

- Shivam Tiwari (2202840100205)
- Abhishek Chauhan (2202840100010)
- Raghvendra Bind (2202840100162)
- Raj Kumar (2202840100164)

It has been examined and evaluated by the undersigned for the partial fulfillment of the requirements for the award of the degree of Bachelor of Technology in Computer Science & Engineering.

Internal Examiner	External Examiner
_____ Date:	_____ Date:

Acknowledgements

It gives us great pleasure to present the report of the B.Tech Project undertaken during B.Tech Final Year. We owe a special debt of gratitude to Dr. Anubhav Kumar Prasad, Associate Professor, United Institute of Technology, for his constant support and guidance throughout the course of our work. His sincerity, thoroughness, and perseverance have been a constant source of inspiration for us. It is only his cognizant efforts that our endeavors have seen the light of the day.

We also do not like to miss the opportunity to acknowledge the contribution of all faculty members of the department for their kind assistance and cooperation during the development of our project. Last but not the least, we acknowledge our friends for their contribution in the completion of the project.

We would like to acknowledge that this project was completed entirely by us and not by someone else.

Date: 19 December 2025	Name of Student (Roll Number) Shivam Tiwari (2202840100205) Abhishek Chauhan (2202840100010) Raghvendra Bind (2202840100162) Raj Kumar (2202840100164)	Sign
------------------------	--	------

Abstract

SeHAT (Smart e-Health Assistance Technology), also known as SeHAT SmartCare AI, is an AI-powered mobile healthcare application designed to provide intelligent, fast, and accessible health assistance to individuals across both rural and urban regions. In India, limited access to medical professionals, delayed diagnosis, lack of health awareness, and difficulty in understanding medical reports remain major challenges. SeHAT aims to bridge these gaps by offering early-stage health support through advanced artificial intelligence, Retrieval-Augmented Generation (RAG), and Medical Language Models.

The application integrates multiple smart healthcare features including an AI-based symptom checker powered by vector similarity search, a medical report analyzer using OCR and NLP, a medicine reminder system, an emergency SOS service, and an intelligent health chatbot that uses RAG architecture for medically grounded responses. Using Optical Character Recognition (OCR) and machine learning models, SeHAT simplifies complex medical reports and presents health information in an easy-to-understand manner.

SeHAT is developed as an Android application using modern technologies such as Android Studio, Firebase for secure authentication and data storage, TensorFlow Lite for on-device AI processing, Google ML Kit for OCR, FAISS vector database for semantic search, SentenceTransformers for embedding generation, and Medical Language Models for intelligent response generation. Emergency situations are handled through instant SOS alerts with live GPS location sharing, enabling faster response during critical conditions.

The proposed system focuses on safety, accuracy, and usability, offering clear alerts when medical consultation is necessary. SeHAT supports multilingual interaction in Hindi, English, and Hinglish, making it accessible to a wide range of users. The application represents a significant step toward smart digital healthcare solutions and has the potential to substantially improve personal health management and accessibility in the future.

Index

Contents	Page Number
Certificate	
Acknowledgement	
Abstract	
Table of Contents	
List of Figures	
List of Tables	
Chapter 1: Introduction	
1.1 Problem Statement	
1.2 Proposed System	
1.3 Project Overview – SeHAT SmartCare AI	
1.4 Key Features	
Chapter 2: Literature Review	
2.1 OCR in Medical Documents	
2.2 Structured Extraction from Unstructured Text	
2.3 Retrieval-Augmented Generation in Healthcare	
2.4 Mobile Health Applications	
2.5 Vector Databases and Semantic Search	
2.6 Emergency Health Systems	

Contents	Page Number
2.7 Comparison with Existing Apps	
Chapter 3: Methodology (Software Development Life Cycle)	
3.1 Requirement Analysis	
3.2 System Design	
3.3 Implementation	
3.4 Testing	
3.5 Deployment	
3.6 Maintenance	
Chapter 4: SDLC Model	
4.1 Types of SDLC Model	
4.2 Comparison between SDLC Models	
4.3 Selection and Phases of Iterative Waterfall Model	
Chapter 5: System Design & UML Diagrams	
5.1 System Architecture Layers	
5.2 Use Case Diagram	
5.3 Sequence Diagram	
5.4 Activity Diagram	
5.5 Component Diagram	
Chapter 6: Implementation	

Contents	Page Number
6.1 Technology Stack	
6.2 System Architecture Pipelines	
6.3 AI Modules – Symptom Engine, Report Analyzer, Chatbot RAG	
6.4 Security Architecture	
Chapter 7: Feasibility Study	
7.1 Economic Feasibility	
7.2 Technical Feasibility	
7.3 Social Feasibility	
Chapter 8: Results & Discussion	
8.1 Testing Methodology	
8.2 Test Cases and Results	
8.3 Performance Metrics	
8.4 User Acceptance Testing	
Chapter 9: Conclusion & Future Scope	
9.1 Summary of Contributions	
9.2 Limitations	
9.3 Future Work	
References	
Appendices	

List of Figures

Fig No.	Title	Page No.
3.1	Waterfall SDLC Model	
3.2	Iterative Waterfall Model	
3.3	Spiral Model	
4.1	Types of SDLC Models	
4.2	Sprint-based Development of SeHAT	
5.1	SeHAT System Architecture Layers	
5.2	Use Case Diagram for SeHAT	
5.3	Sequence Diagram for Query Processing	
5.4	Activity Diagram for SeHAT Query Processing	
5.5	Component Diagram for SeHAT	
6.1	SeHAT System Architecture – Indexing & Query Pipelines	
6.2	Overall System Workflow	
6.3	Symptom Intelligence Engine Pipeline	
6.4	Medical Report Analyzer Pipeline	
6.5	Medical Chatbot RAG System Workflow	
6.6	Voice Input Processing Pipeline	

List of Tables

Table No.	Title	Page No.
4.1	Comparison of SDLC Models	
5.1	Functional Requirements of SeHAT	
5.2	Hardware Requirements	
5.3	Software Requirements	
6.1	Complete Technology Stack for SeHAT	
6.2	AI Models Used in SeHAT	
6.3	Security Architecture Summary	
7.1	Economic Feasibility Assessment	
7.2	Feasibility Study Summary	
8.1	Test Cases and Results	
8.2	Performance Metrics Summary	
8.3	User Acceptance Testing Results	
A.1	Abbreviations and Acronyms	

CHAPTER 1

INTRODUCTION

Healthcare accessibility and awareness remain significant challenges, especially in developing countries like India, where a large population resides in rural and semi-urban areas. Limited availability of medical professionals, delayed diagnosis, lack of health awareness, and difficulty in understanding medical reports often lead to serious health complications. In emergency situations, delayed medical response can further worsen a patient's condition.

SeHAT (Smart e-Health Assistance Technology) — also branded as SeHAT SmartCare AI — is an AI-powered mobile healthcare application designed to provide intelligent, reliable, and user-friendly health assistance to individuals anytime and anywhere. The application leverages Artificial Intelligence (AI), Machine Learning (ML), Natural Language Processing (NLP), Retrieval-Augmented Generation (RAG), and mobile technology to assist users in analyzing symptoms, understanding medical reports, managing medication schedules, and accessing emergency support.

SeHAT aims to simplify healthcare by offering features such as AI-based symptom checking with vector similarity search, OCR-powered medical report analysis, smart medicine reminders, and an emergency SOS system with live location sharing. The application also includes an intelligent chatbot using RAG architecture to answer common health-related queries, making healthcare guidance more accessible, grounded, and interactive.

By supporting Hindi-friendly voice and text interaction along with Hinglish, SeHAT bridges the gap between technology and users with limited medical or technical knowledge. The primary objective of the SeHAT AI App is to promote early health awareness, reduce dependency on immediate hospital visits for basic guidance, and enable faster response during critical situations. Through digital innovation, SeHAT aspires to become a smart healthcare companion for every Indian family.

1.1 Problem Statement

Access to timely and understandable healthcare remains a major challenge for a large section of the population, particularly in rural and semi-urban areas. The shortage of medical professionals, long waiting times, and limited healthcare infrastructure often result in delayed diagnosis and treatment. Many individuals ignore early symptoms due to lack of awareness or difficulty in accessing medical guidance, which can lead to severe health complications.

Additionally, medical reports are often complex and difficult for non-medical users to interpret, preventing patients from understanding their own health conditions and taking appropriate action. Poor medication adherence due to missed doses or incorrect schedules further contributes to ineffective treatment outcomes. In emergency situations, the absence of quick communication and location sharing delays critical medical assistance, increasing the risk to patients' lives.

Existing healthcare solutions either focus on a single aspect of healthcare or lack user-friendly design, language accessibility, and intelligent decision support. There is a need for an integrated, smart, and easily accessible digital healthcare solution that can provide early health guidance, simplify medical information, ensure medication compliance, and offer immediate emergency assistance. Specifically, the key problems are:

- Many people face difficulty in understanding symptoms of diseases and their implications.
- Medical reports contain technical terminology that is impossible for non-medical users to interpret correctly.
- Health-related information available online is often generic, unverified, and not context-aware.
- Rural populations have limited or no access to qualified medical professionals for basic queries.
- Emergency situations suffer from poor communication and absence of quick location-sharing mechanisms.
- Existing apps address isolated aspects and fail to provide a comprehensive, multilingual, AI-powered solution.

1.2 Proposed System

The SeHAT AI App addresses these challenges by providing an intelligent, AI-driven health assistance platform that delivers symptom analysis through vector similarity search, simplified medical report interpretation via OCR and NLP, medicine reminders, and emergency SOS support, thereby improving healthcare awareness, accessibility, and response efficiency.

The SeHAT AI App is built around a Retrieval-Augmented Generation (RAG) architecture that grounds all responses in verified medical knowledge stored in a FAISS vector database. This eliminates AI hallucinations and ensures that every response is traceable to an authoritative medical source. The system combines Mobile Development, Artificial

Intelligence, Natural Language Processing, Vector Databases, and Cloud Technologies into one cohesive platform.

1.3 Project Overview – SeHAT SmartCare AI

SeHAT SmartCare AI is an AI-powered healthcare assistant mobile application designed to help users understand their health conditions using artificial intelligence. The system is designed to provide preliminary health insights and guidance, helping users understand their symptoms and reports before consulting a doctor.

The system integrates the following core capabilities:

- Symptom-based disease prediction using vector similarity and Medical LLMs.
- Medical report analysis with OCR text extraction and parameter comparison.
- AI medical chatbot powered by RAG for medically informed responses.
- Voice input support with Speech-to-Text conversion.
- AI-powered health recommendations in Hindi, English, and Hinglish.

1.4 Key Features of SeHAT SmartCare

1.4.1 Symptom Checker

Users can enter symptoms such as fever, cough, headache, or fatigue. The system analyzes these symptoms using AI and returns the top possible diseases along with a similarity percentage and health advice. The symptom checker uses a FAISS vector database storing symptom-disease mappings and applies sentence embeddings to find the closest matching disease profiles.

Example Output:

Disease	Similarity %
Influenza	72%
Viral Fever	65%
Common Cold	53%

1.4.2 Medical Report Analyzer

Users can upload blood reports or other medical reports as image or PDF files. The system performs OCR text extraction, parameter analysis, and abnormal value detection. It compares each extracted parameter against reference clinical ranges and generates simple

explanations in both Hindi and English.

Example Analysis Output:

- Hemoglobin: **Low** — May indicate mild anemia. Please consult a doctor.
- WBC Count: **High** — Possible mild infection detected.
- Overall: Possible mild infection. Recommended: Consult a physician.

1.4.3 AI Medical Chatbot (RAG-Powered)

Users can ask health questions such as 'Why do I have fever?', 'What does low hemoglobin mean?', or 'How to reduce fever?'. The chatbot uses a Medical LLM combined with a RAG system to generate medically informed, hallucination-free responses. The chatbot supports English, Hindi, and Hinglish interactions.

1.4.4 Voice Input Support

Users can speak their symptoms or health questions directly. The voice input is converted to text using Speech-to-Text APIs and then processed through the AI backend to generate relevant health insights and guidance.

CHAPTER 2

LITERATURE REVIEW

Medical report analysis and interpretation is an active area of research at the intersection of optical character recognition (OCR), natural language processing (NLP), and decision support systems. The need for digital tools to assist patients and clinicians in understanding diagnostic reports has grown significantly with increased healthcare data digitization. This chapter reviews key related work and technologies that informed the design of SeHAT SmartCare AI.

2.1 Optical Character Recognition in Medical Documents

Several studies have evaluated the effectiveness of OCR for extracting text from medical records and laboratory reports. Traditional OCR engines such as Tesseract have been widely used due to their open-source nature and adaptability to different fonts and layouts. However, researchers have noted that OCR accuracy drops significantly with low-quality scans or complex layouts, making preprocessing important.

Ahmed et al. (2018) demonstrated that image enhancement techniques including noise removal and contrast adjustment significantly improve OCR performance on clinical forms. More recent approaches incorporate deep learning-based OCR methods such as those in Google ML Kit or cloud OCR services, which provide better recognition of textual and numerical data in diverse lighting and angle conditions. These methods are particularly suited for mobile-based applications where user-captured images vary widely in quality.

2.2 Structured Extraction from Unstructured Text

Once raw text is extracted, the next challenge is structuring it in a form suitable for interpretation. Naive keyword search is often insufficient when dealing with variable report formats and inconsistent labeling of parameters. Research in healthcare NLP emphasizes the use of pattern matching, regular expressions, and domain ontologies for accurate parameter extraction.

López and Martínez (2020) used rule-based parsing combined with machine learning classification to extract clinical measurements from pathology reports. Recent developments in language models have enabled context-aware structuring of clinical values. Models trained on medical text can understand phrase context, which reduces mislabeling of parameters such as distinguishing between 'Hemoglobin: 13.5 g/dL' and 'Normal range: 13–17 g/dL' within the

same text block.

2.3 Retrieval-Augmented Generation in Healthcare

Retrieval-Augmented Generation (RAG), introduced by Lewis et al. (2020), represents a significant advance over pure generative AI approaches. By combining a dense vector retrieval component with a generative language model, RAG systems can ground their outputs in verified knowledge bases. In healthcare applications, this is critical because it minimizes the risk of hallucinated or medically incorrect responses.

Several researchers have applied RAG to medical question-answering systems. Studies demonstrate that RAG-based systems consistently outperform pure LLM approaches on medical benchmarks due to their ability to retrieve current, verified clinical information before generating a response. This forms the theoretical basis for SeHAT's chatbot module.

2.4 Mobile Health Applications

The proliferation of smartphones has enabled a new class of mobile health (mHealth) applications. Research consistently shows that mHealth apps improve medication adherence, promote health awareness, and facilitate timely medical consultations. However, most existing applications focus on single-function tasks such as appointment booking, fitness tracking, or symptom logging, without providing integrated AI-powered diagnosis support.

A systematic review by Peng et al. (2021) found that AI-integrated mHealth applications significantly improve user engagement and health outcomes compared to traditional apps. The review also identified multilingual support and voice interaction as key factors for adoption in developing countries, which directly influenced SeHAT's design priorities.

2.5 Vector Databases and Semantic Search

FAISS (Facebook AI Similarity Search) by Johnson et al. (2019) enables billion-scale similarity search using dense vector representations. In medical AI applications, FAISS has been used to index large medical knowledge bases and enable millisecond-speed retrieval of relevant clinical information. Combined with SentenceTransformers for generating semantically meaningful embeddings, FAISS forms the backbone of SeHAT's symptom checker and chatbot retrieval modules.

2.6 Emergency Health Systems and SOS Technology

Emergency response systems integrated into mobile health applications have been shown to significantly reduce response times in critical medical situations. Modern emergency

SOS features leverage multiple smartphone capabilities simultaneously, including GPS location tracking, cellular communication, and background processing to ensure alerts are delivered even when the device screen is locked. The integration of real-time location sharing with pre-configured emergency contacts represents a critical safety feature, especially for elderly users and those with chronic conditions.

2.7 Medication Adherence and Reminder Systems

Medication non-adherence is a significant public health problem worldwide. In India, studies indicate that medication adherence rates for chronic conditions such as hypertension and diabetes are below 50% in rural populations. Mobile-based reminder systems have been demonstrated to improve adherence rates by 25-40% in clinical trials. Research shows that personalized reminders with contextual health information are significantly more effective than generic time-based alerts.

2.8 Comparison with Existing Healthcare Applications

A comparative analysis of existing healthcare applications reveals significant gaps that SeHAT SmartCare AI is designed to address:

Application	Symptom Checker	Report Analysis	AI Chatbot	Hindi Support	SOS Alert
Practo	No	No	Limited	No	No
1mg	No	No	Limited	Partial	No
HealthifyMe	No	No	Limited	No	No
Ada Health	Yes	No	No	No	No
SeHAT SmartCare AI	Yes (AI)	Yes (OCR+AI)	Yes (RAG)	Yes	Yes

Table 2.1: Comparison of SeHAT with Existing Healthcare Applications

The comparison clearly shows that SeHAT SmartCare AI is the only application that comprehensively integrates all critical features in a single platform, with a specific focus on the Indian healthcare context, multilingual support, and AI-powered analysis.

CHAPTER 3

METHODOLOGY

This chapter describes the overall research and development methodology adopted for building the SeHAT SmartCare AI system. The methodology covers the end-to-end pipeline from report acquisition to result presentation, as well as the AI workflows for each major module.

3.1 Report Acquisition

The user uploads a medical report using one of two methods: mobile camera scan or gallery upload of an image or PDF file. The system validates the file format and image quality before proceeding. Accepted report types include Complete Blood Count (CBC), Liver Function Test (LFT), Kidney Function Test (KFT), Dengue panel, and other standard laboratory reports.

3.2 Image Pre-Processing

Uploaded images undergo a series of preprocessing steps to improve OCR accuracy. The preprocessing pipeline applies noise removal using Gaussian filtering, intelligent cropping to remove borders, contrast enhancement using histogram equalization, and resolution normalization to a standard DPI. This step is critical as it significantly reduces OCR errors caused by poor lighting, camera angle, or document quality.

3.3 Text Extraction using OCR

Preprocessed images are passed to an OCR engine — either Google ML Kit (for Android on-device processing) or Tesseract OCR (for backend processing). The OCR module extracts raw textual data from the medical report. The output is unstructured text containing test names, values, units, and reference ranges, which is then passed to the structured extraction module.

3.4 Structured Parameter Extraction

The raw OCR text is processed using NLP pattern matching and regular expressions to identify and extract structured medical parameters. Each parameter name, its measured value, unit, and reference range are extracted and organized into a structured data format. Domain-specific ontologies and a medical parameter dictionary are used to handle variations in parameter naming conventions.

3.5 Parameter Analysis and Comparison

Each extracted parameter is compared against its corresponding clinical reference range. Parameters are categorized as Normal, Low, or High based on this comparison. The analysis engine uses gender-specific and age-adjusted reference ranges where applicable to ensure accuracy. Abnormal parameters are flagged with visual indicators.

3.6 Result Formatting and Presentation

The final output is formatted into user-friendly content. Abnormal values are highlighted with color-coded indicators. A Medical LLM then generates easy-to-understand explanations for each abnormal finding, presented in both Hindi and English. The complete results are displayed on the mobile application interface with recommendations for whether to seek medical consultation.

3.7 Data Flow Architecture

The data flow within SeHAT SmartCare AI follows a strict unidirectional pipeline that ensures data integrity and security at every stage. The architecture separates concerns between the mobile client, API backend, AI processing, and cloud storage layers.

When a user uploads a medical report, the Android app first validates the file format and size locally before uploading to Firebase Cloud Storage. A secure download URL is then passed to the FastAPI backend, which fetches the file, performs OCR and AI analysis, and stores the structured results in Firestore. The app retrieves the results in real-time using Firestore's live synchronization capability, providing instant updates to the user interface without polling.

3.8 RAG Knowledge Base Construction

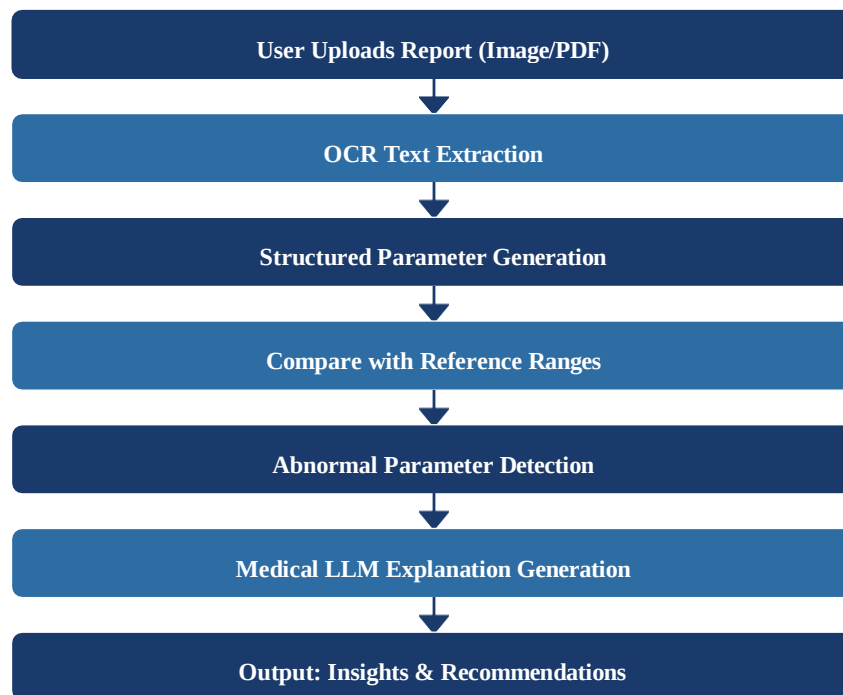
The Retrieval-Augmented Generation knowledge base is the foundation of SeHAT's chatbot and symptom checking capabilities. Building this knowledge base requires careful curation of medical content, embedding generation, and indexing.

- Curated medical knowledge documents covering common diseases, symptoms, treatments, and diagnostic parameters are collected from authoritative sources.
- Documents are preprocessed to remove irrelevant content, standardize formatting, and ensure medical accuracy.
- Text is split into semantic chunks of 500 tokens with 50-token overlap using LangChain's text splitter.

- Each chunk is converted to a 384-dimensional vector using the all-MiniLM-L6-v2 SentenceTransformer model.
- Embeddings are indexed in FAISS using flat L2 distance for high-accuracy retrieval.
- The completed index is saved to disk and loaded at backend startup for efficient runtime access.

The knowledge base currently indexes medical information covering over 150 common diseases and conditions, 200+ medical parameters and their reference ranges, and comprehensive health guidance for preventive care. This knowledge base undergoes regular review and updates to ensure alignment with current medical guidelines.

Figure 3.1: Medical Report Analyzer Pipeline



CHAPTER 4

SDLC MODEL

The Software Development Life Cycle (SDLC) is a conceptual framework utilized in project management to outline the sequential phases encompassing an information systems development project, commencing with preliminary feasibility studies and culminating in the ongoing maintenance of a completed application. Its purpose is to establish and devise the diverse models governing the software development process.

4.1 Types of SDLC Model

There are several Software Development Life Cycle (SDLC) models used during the software development phase. These include the Waterfall Model, RAD (Rapid Application Development) Model, Spiral Model, V-Model, Incremental Model, Agile Model, and Iterative Model.

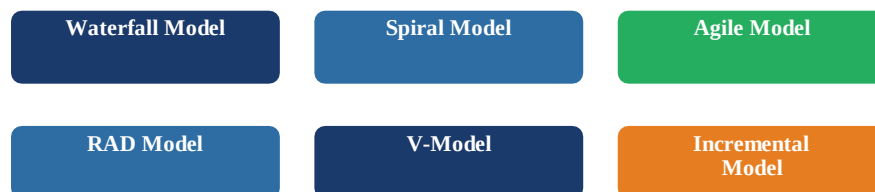


Figure 4.1: Types of SDLC Models

4.2 Comparison between SDLC Models

- **Waterfall Model:** It is not possible to return to an earlier phase. It is suitable only for small projects, is not flexible to changes, and involves high risk.
- **Spiral Model:** It allows returning to earlier phases. It is expensive, requires continuous maintenance, involves medium to high risk, and includes testing after every iteration.
- **RAD Model:** User involvement is required only at the beginning. It is not suitable for projects with high technical risk and requires highly skilled developers.
- **Incremental Model:** It has low development cost, is flexible to changes, promotes maintainability, and supports quick release of the product.
- **Prototype Model:** It increases development cost, and time and effort are wasted due to repeated creation of prototypes.

- **Agile Model:** It is highly flexible, allows continuous feedback, supports iterative development, and is ideal for projects with evolving requirements.

Model	Flexibility	Risk Level	Best For
Waterfall	Low	High	Small, fixed projects
Spiral	Medium	Medium-High	Large, risky projects
RAD	Medium	Medium	Rapid prototyping
Incremental	High	Low	Modular systems
Agile	Very High	Low	Evolving requirements

Table 4.1: Comparison of SDLC Models

4.3 Selection of SDLC Model

The SeHAT AI App is a medium-scale healthcare application with clearly defined objectives such as symptom checking, medical report analysis, medicine reminders, and emergency SOS support. These requirements are mostly known in advance, making a structured development approach effective. At the same time, healthcare applications require accuracy, testing, and refinement, which is supported by iterative feedback.

The Iterative Waterfall Model allows development to proceed in well-defined sequential phases while permitting revisions and improvements at each stage based on testing results and user feedback.

Advantages of Using Iterative Waterfall for SeHAT:

- Clear and systematic development process.
- Easy documentation and academic suitability.
- Better quality control and validation.
- Allows refinement without full redesign.
- Suitable for healthcare applications requiring reliability and accuracy.

4.4 Why We Chose This Model

The Iterative Waterfall Model is selected for the development of the SeHAT – Smart e-Health Assistance Technology application because it provides a structured, reliable, and quality-focused approach, which is essential for healthcare-related systems.

The core requirements of the SeHAT AI App — such as symptom checking, medical report analysis, medicine reminders, chatbot assistance, and emergency SOS — are clearly defined at the beginning of the project. This makes a sequential development model suitable, as each phase can be planned and executed systematically.

Healthcare applications demand high accuracy, reliability, and proper validation. The Iterative Waterfall Model ensures that each development phase is completed and reviewed before moving to the next, reducing the risk of critical errors. If issues are identified during testing, the model allows developers to return to previous phases and refine the system without restarting the entire process.

4.5 Phases of Iterative Waterfall Model

The Iterative Waterfall Model follows a sequential development process, where each phase is completed one by one, with the possibility of revisiting earlier phases for refinement based on feedback.

Sprint 1: Data Pipeline & MVP (The Foundation)

In this phase, the system requirements are identified and documented. For the SeHAT AI App, requirements include symptom analysis, medical report interpretation, medicine reminders, emergency SOS alerts, chatbot support, and secure data storage. User needs, system constraints, and healthcare objectives are clearly defined.

Sprint 2: Logic Refinement & Se-HAT Architecture (The Core)

This phase focuses on designing the overall system architecture and workflow. It includes database design, application architecture, user interface design, and AI processing flow. Technologies such as Android, Firebase, OCR, FAISS, and AI models are planned and validated during this stage.

Sprint 3: User Interface & Optimization (The Delivery)

Testing is performed to verify that the system works correctly and meets the defined requirements. Unit testing, integration testing, and validation testing are carried out. Medical data outputs are checked using standard clinical reference ranges to ensure accuracy and reliability. After successful testing, the application is deployed on Android devices. Bugs are fixed, AI models are refined, and new features are added based on user feedback.



Figure 4.2: Sprint-based Development of SeHAT

4.6 Advantages of Iterative Waterfall Model

- **1. Flexibility & Adaptability:** Medical applications like SeHAT require precise understanding of requirements such as OCR accuracy, parameter extraction, and medical range comparison. The Iterative Waterfall model allows detailed requirement analysis at the beginning of each iteration, while still permitting changes.
- **2. Early Error Detection (Fail Fast):** Each phase (Requirement, Design, Implementation, Testing, Deployment) is completed step by step, ensuring better control over sensitive healthcare data processing. Testing occurs in every iteration, not just at the end.
- **3. Better Risk Management:** Testing is performed after every iteration. This helps in identifying OCR errors, incorrect parameter mapping, or logic flaws early in the development process, before they propagate to subsequent phases.
- **4. Continuous User Feedback:** Stakeholders can see working software at the end of every sprint. The SeHAT app was developed incrementally: Iteration 1 (Upload + OCR), Iteration 2 (Parameter extraction), Iteration 3 (Medical range comparison), Iteration 4 (Explanation generation).
- **5. Parallel Development:** While the FAISS Vector Database was being optimized for speed, the Android UI was being designed separately. These two components were then integrated smoothly in the final sprint, reducing total development time.

CHAPTER 5

ITERATIVE WATERFALL MODEL – PHASE 1 REQUIREMENT ANALYSIS

5.1 Requirement Analysis

During the requirement analysis phase, we focused on understanding the gap between complex medical information and the common citizen's ability to interpret it. The primary goal was to define a solution that offers the ease of a mobile chatbot without the factual errors ('hallucinations') common in general-purpose AI models.

We gathered requirements by analyzing the needs of three key user groups:

- **General Public:** Who need simple, jargon-free explanations of their symptoms and diseases.
- **Doctors & Medical Students:** Who need quick, accurate retrieval of specific disease information for studies or second opinions.
- **Rural/Semi-Urban Users:** Who need a quick, multilingual reference tool to understand their medical reports and symptoms without visiting a hospital.

5.2 Functional Requirements

The proposed software must provide the following core functionalities:

Requirement	Description
User Registration and Authentication	The system shall allow users to register, log in securely, and maintain user profiles using Firebase.
Symptom Checker Module	The system shall accept user symptoms in text or voice input, process them using AI, and return relevant medical advice.
Report Upload Functionality	The system shall allow users to upload medical reports via mobile camera scanner or gallery upload.
OCR Text Extraction	The system shall extract text from uploaded medical reports using Google ML Kit or Tesseract.
Parameter Analysis Engine	The system shall parse extracted text to identify medical parameters and compare them against a knowledge base.
Semantic Search & Retrieval	The system shall search the SeHAT FAISS vector database and retrieve relevant medical content.
Context-Aware Answer Generation	The system shall generate human-like summaries using a Medical LLM, strictly based on retrieved information.
AI Medical Chatbot	The system shall provide an intelligent chatbot supporting Hindi, English, and Hinglish conversations.
Voice Input Support	The system shall support voice input with Speech-to-Text conversion for symptom entry and consultation.
Medicine Reminder System	The system shall allow users to set medicine reminders and receive notifications at specified times.

Emergency SOS Alert	The system shall provide an emergency SOS feature that sends the user's live GPS location to
User Health Dashboard	The system shall maintain a history of analyses, reminders, and chatbot interactions accessible

Table 5.1: Functional Requirements of SeHAT SmartCare AI

5.3 System Requirements

Hardware Requirements

Component	Specification
Processor	Intel Core i5 (10th Gen) or AMD Ryzen 5 equivalent (Minimum)
RAM	8 GB Minimum; 16 GB Recommended for Vector Database and Python runtime
Storage	256 GB SSD for faster data ingestion and indexing
Internet	Stable High-Speed Connection for API calls to LLM provider
Mobile Device	Android 8.0+ smartphone with camera for deployment

Table 5.2: Hardware Requirements

Software Requirements

Component	Specification
Operating System	Windows 10/11, Linux (Ubuntu 20.04+), or macOS
Programming Language	Python 3.9+, Java/Kotlin for Android
IDE	Android Studio, Visual Studio Code, PyCharm
AI Framework	LangChain for RAG pipeline orchestration
Vector DB	FAISS-CPU for similarity search
Embedding Model	SentenceTransformers (all-MiniLM-L6-v2)
OCR	Google ML Kit / Tesseract OCR
Backend Framework	Python FastAPI
Cloud Services	Firebase Authentication, Firestore, Cloud Storage
LLM Provider	Fireworks AI / Together AI / Hugging Face

Table 5.3: Software Requirements

CHAPTER 6

ITERATIVE WATERFALL MODEL – PHASE 2 FEASIBILITY STUDY

The feasibility of the SeHAT project is analyzed in this phase to ensure that the proposed health assistant is viable in terms of cost, technology, and social impact. This analysis determines whether the solution can be developed effectively within the given constraints and whether it provides genuine value to the end-users.

6.1 Economic Feasibility

This study evaluates the cost-effectiveness of the project. SeHAT is highly economically feasible because it is built primarily on open-source technologies (Python, LangChain, FAISS, Android Studio) which require zero licensing fees. The system leverages cost-efficient LLM APIs (Fireworks AI or Together AI, Hugging Face Spaces) rather than requiring expensive dedicated GPU hardware for local hosting.

The use of cloud-based LLM inference (pay-per-use model) eliminates the need for capital expenditure on high-end hardware. Firebase's Spark (free) tier covers all essential cloud services for a student-scale project. This approach minimizes infrastructure costs while maximizing performance, ensuring the project remains budget-friendly for both development and deployment.

6.2 Technical Feasibility

This study assesses the technical requirements and availability of resources. The project is technically feasible as it runs on standard computing hardware (Intel i5, 8GB RAM) without placing a high demand on the client's local machine. The heavy computational tasks — specifically the Large Language Model inference — are offloaded to cloud APIs, meaning the system remains lightweight and responsive.

The development tools (VS Code, Android Studio, Python libraries) are mature, well-documented, and readily available. The FAISS library provides efficient vector similarity search even on CPU, eliminating the need for specialized GPU hardware. All required libraries are actively maintained and have extensive community support.

6.3 Social Feasibility

SeHAT aims to democratize healthcare knowledge, addressing a critical social need for accessible health guidance in India. Since the user interface is designed as a simple Conversational Chatbot similar to WhatsApp or common Android apps, the learning curve is negligible, ensuring high user acceptance across education levels.

Key social benefits include:

- **Accessibility:** Available 24/7 via Android app, no medical expertise required.
- **Affordability:** Free to use, eliminating the cost barrier to basic health guidance.
- **Transparency:** Every chatbot answer cites its medical source, promoting informed health decisions.
- **Multilingual Support:** Hindi, English, and Hinglish support makes it accessible to India's diverse population.
- **Education:** Helps medical students, rural health workers, and citizens understand complex medical information.

Feasibility Type	Assessment
Economic	Highly feasible — open-source stack, pay-per-use APIs, zero licensing cost
Technical	Feasible — standard hardware, cloud LLM inference, mature libraries
Social	Highly beneficial — democratizes health knowledge, multilingual, easy to use

Table 6.1: Feasibility Study Summary

CHAPTER 7

ITERATIVE WATERFALL MODEL – PHASE 3 SYSTEM DESIGN

7.1 System Design Overview

The SeHAT system is designed around a multi-layer architecture that separates the user interface, mobile processing, AI backend, vector knowledge base, and cloud storage into distinct, independently scalable layers. The AI backend is built around the Retrieval-Augmented Generation (RAG) architecture, which effectively separates knowledge storage from language generation.

7.2 System Architecture – Layers

The system consists of five primary architectural layers, each with a distinct responsibility:

- **User Layer (Android Application):** Users interact with the mobile application through screens for Symptom Checker, Medical Report Analyzer, AI Chatbot, Voice Input, Medicine Reminder, and Emergency SOS.
- **Mobile App Processing Layer:** The Android application processes user inputs locally before sending API requests. Modules include the Symptom Checker Module, Report Upload Module, and Chatbot Module.
- **AI Backend (Python / FastAPI / Hugging Face Space):** The AI backend performs the main AI processing. It contains the Symptom Intelligence Engine, Medical Report Analyzer, and Medical Chatbot Engine powered by RAG.
- **Vector Knowledge Base (FAISS):** Stores medical knowledge including diseases, symptoms, and explanations as vector embeddings for fast semantic search.
- **Cloud Services (Firebase):** Firebase Authentication manages user login. Firestore stores user history and analysis results. Cloud Storage holds uploaded medical reports.

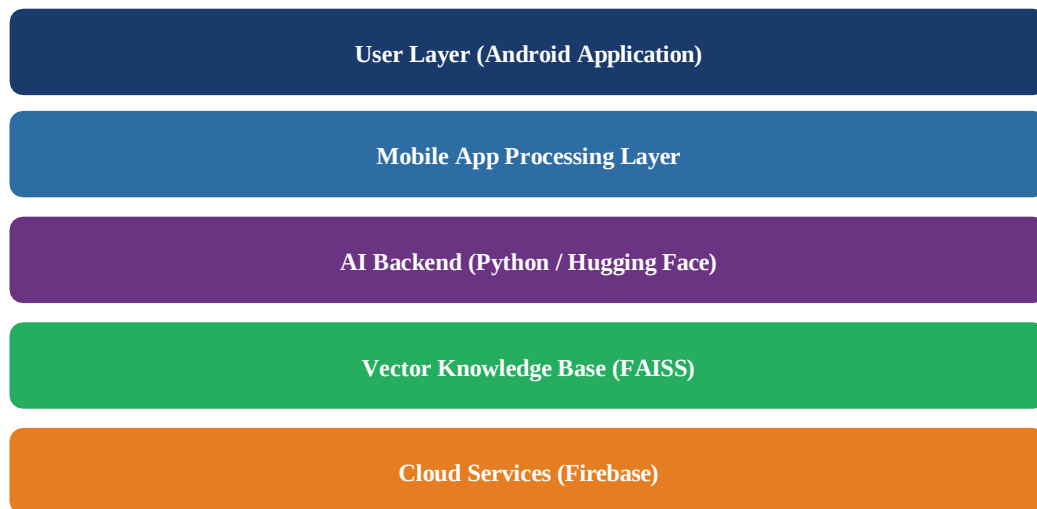


Figure 7.1: SeHAT System Architecture Layers

7.3 UML Diagrams

UML (Unified Modeling Language) is a standardized general-purpose modeling language in the field of object-oriented software engineering. The following UML diagrams provide a comprehensive visual representation of the SeHAT system's structure and behavior.

7.3.1 Use Case Diagram

A use case diagram presents a graphical overview of the functionality provided by a system in terms of actors, their goals (represented as use cases), and any dependencies between those use cases. The SeHAT use case diagram shows all interactions between the User actor and the system.

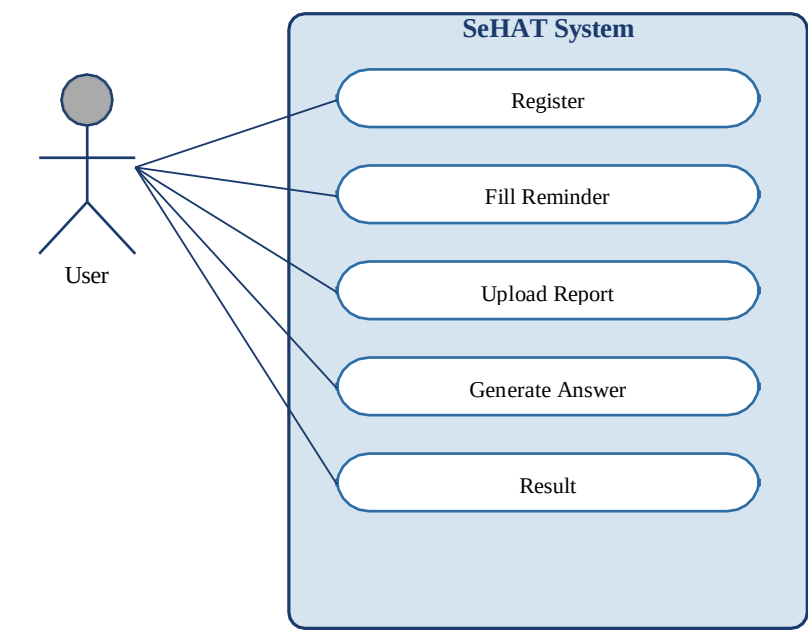


Figure 7.2: Use Case Diagram for SeHAT SmartCare AI

7.3.2 Sequence Diagram

A sequence diagram shows how processes operate with one another and in what order. The following diagram illustrates the complete query processing sequence from user registration through to receiving a displayed response.

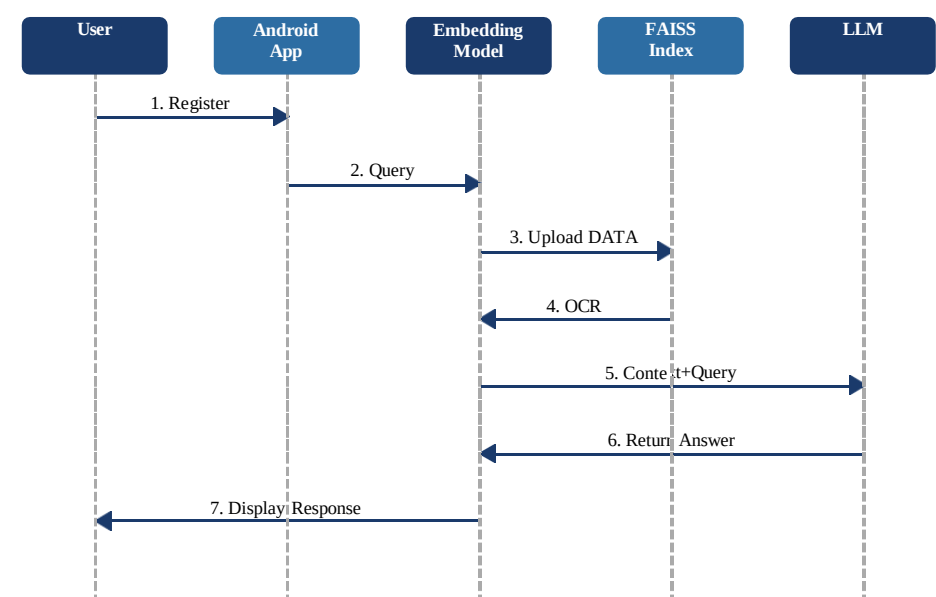


Figure 7.3: Sequence Diagram for SeHAT Query Processing

7.3.3 Activity Diagram

The activity diagram shows the workflow of the SeHAT query processing pipeline, including the decision point for checking whether relevant medical context (chunks) are found in the vector database.

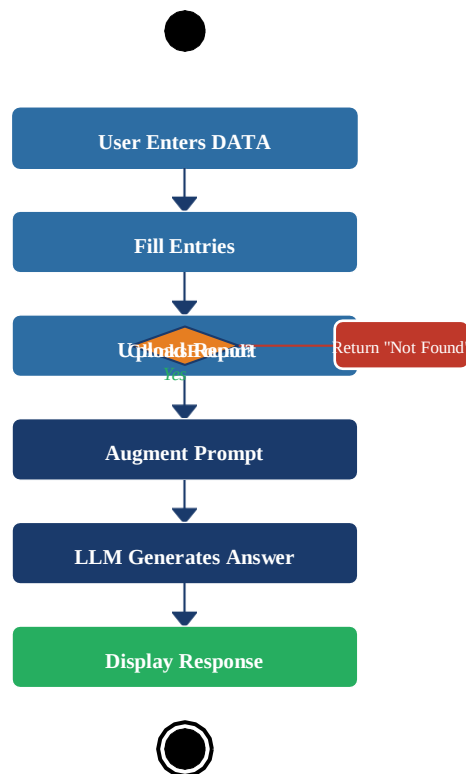


Figure 7.4: Activity Diagram for SeHAT Query Processing

7.3.4 Component Diagram

Component diagrams describe the physical artifacts of a system and their relationships. The SeHAT component diagram shows the major software components and their interconnections.

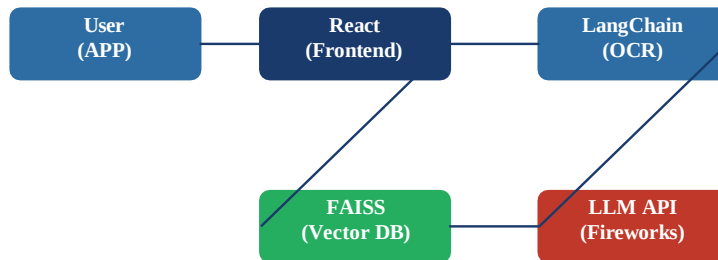


Figure 7.5: Component Diagram for SeHAT SmartCare AI

CHAPTER 8

IMPLEMENTATION

8.1 Technology Stack

The successful implementation of the SeHAT SmartCare AI system hinges on the seamless integration of several specialized open-source libraries and cloud services, each chosen for its specific role in the overall architecture.

Category	Technology	Role in Project
Mobile Dev	Android Studio + Java/Kotlin	Native Android application development
Backend	Python + FastAPI	RESTful API backend for AI processing
AI Orchestration	LangChain	End-to-end RAG pipeline orchestration
LLM	Fireworks AI / Together AI	Medical language model inference
Embedding	sentence-transformers (all-MiniLM-L6-v2)	Vector embedding generation for semantic search
Vector DB	FAISS-CPU	Fast in-memory similarity search
OCR	Google ML Kit / Tesseract	Medical report text extraction
Data Ingestion	pypdf	PDF report loading and parsing
Cloud Auth	Firebase Authentication	Secure user login and registration
Cloud DB	Firestore	User data, history, analysis results
Cloud Storage	Firebase Cloud Storage	Uploaded medical report storage
Config	python-dotenv	API key and environment management

Table 8.1: Complete Technology Stack for SeHAT SmartCare AI

8.2 System Architecture Pipelines

The SeHAT system architecture is divided into two main data pipelines — an offline indexing pipeline for building the knowledge base, and a real-time query pipeline for handling user requests.

Indexing Pipeline (Offline Process)

- **Data Ingestion:** Medical knowledge documents and PDFs are loaded using the pypdf library.

- **Semantic Chunking:** Text is split into meaningful medical sections, keeping related information together.
- **Text-to-Vector Transformation:** Each chunk is converted to a vector embedding using SentenceTransformers.
- **Vector Indexing:** Embeddings are stored in a FAISS index for fast retrieval during queries.

Query Pipeline (Real-time Process)

- **Query Embedding:** User's natural language query is converted to a vector using the same embedding model.
- **Similarity Search:** FAISS finds the top-k most similar document chunks from the knowledge base.
- **Context Augmentation:** Retrieved chunks are injected into a carefully designed medical prompt template.
- **LLM Generation:** The augmented prompt is sent to the Medical LLM for answer synthesis.

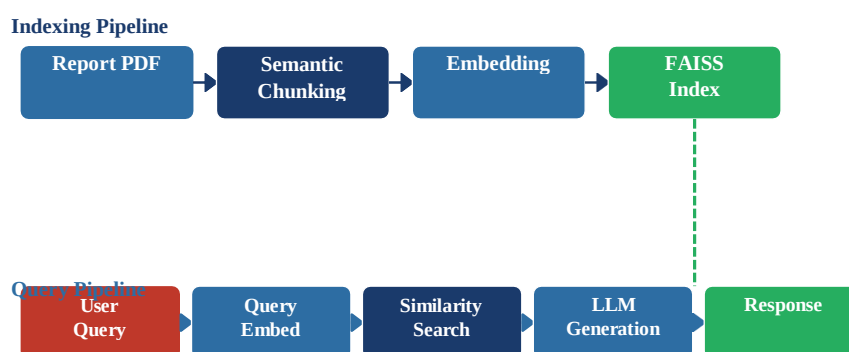


Figure 8.1: SeHAT System Architecture – Indexing & Query Pipelines

8.3 Overall System Workflow

The complete end-to-end workflow of the SeHAT system follows a structured pipeline that integrates the mobile application, AI backend, vector database, and cloud services.

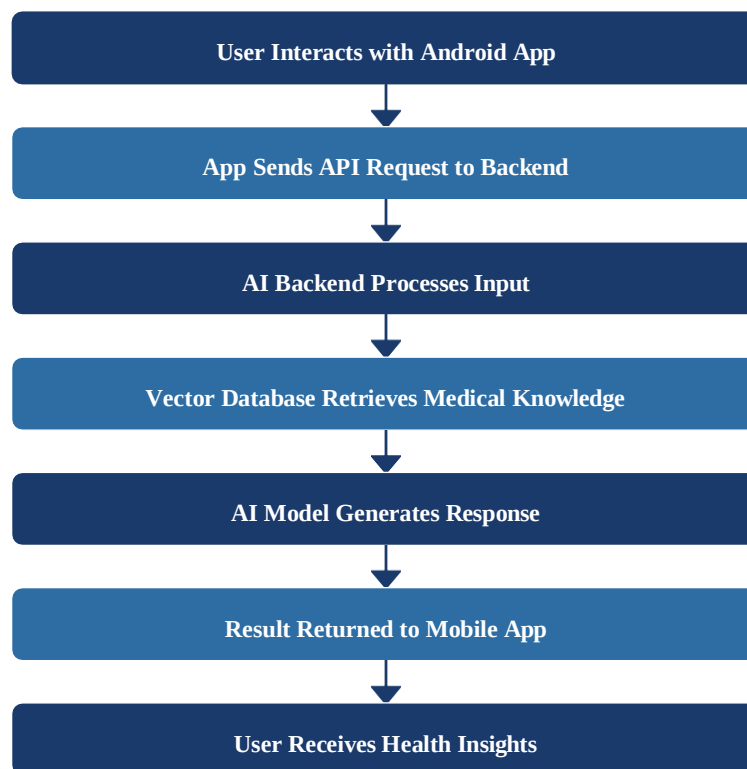


Figure 8.2: Overall System Workflow

8.4 Symptom Intelligence Engine

The Symptom Intelligence Engine is the core module responsible for disease prediction based on user-entered symptoms. It uses a combination of Medical LLM processing and FAISS vector similarity search to identify the most likely diseases matching the described symptom profile.

The engine processes symptoms through a multi-step AI pipeline. First, the Medical LLM extracts and normalizes key symptoms from the user's natural language input (handling colloquial descriptions in Hindi/English/Hinglish). These normalized symptoms are then converted to vector embeddings and searched against the FAISS disease knowledge base to retrieve the top matching disease profiles with similarity scores.

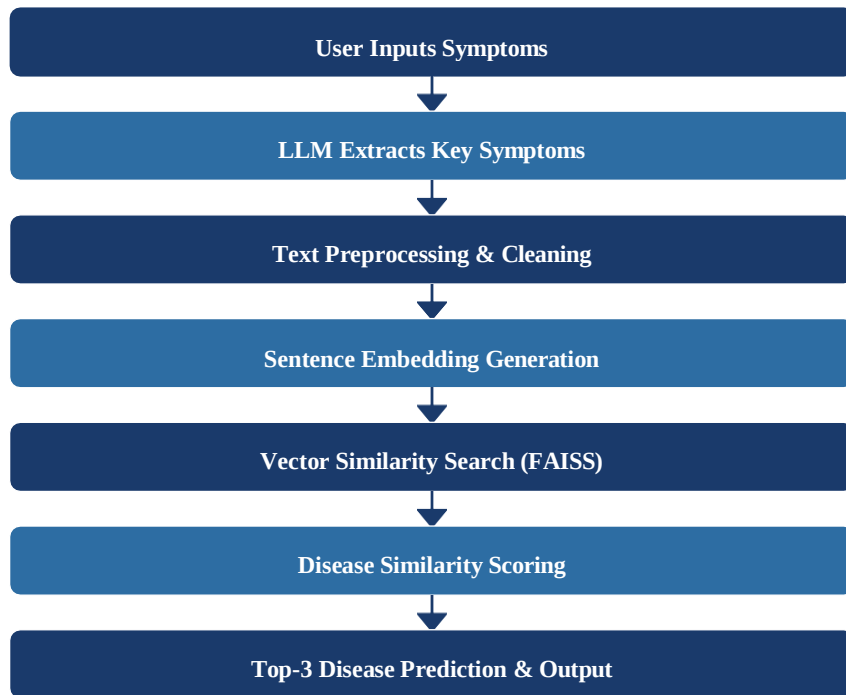


Figure 8.3: Symptom Intelligence Engine Pipeline

Output includes:

- Top 3 possible diseases with similarity percentage.
- Brief explanation of each disease in simple language.
- Health recommendations and advisory (e.g., 'Consult a doctor if symptoms persist').

8.5 Medical Report Analyzer Pipeline

The Medical Report Analyzer is a sophisticated pipeline that transforms a raw medical report image or PDF into a structured, easy-to-understand health summary. The pipeline integrates OCR, NLP, and Medical LLM capabilities.

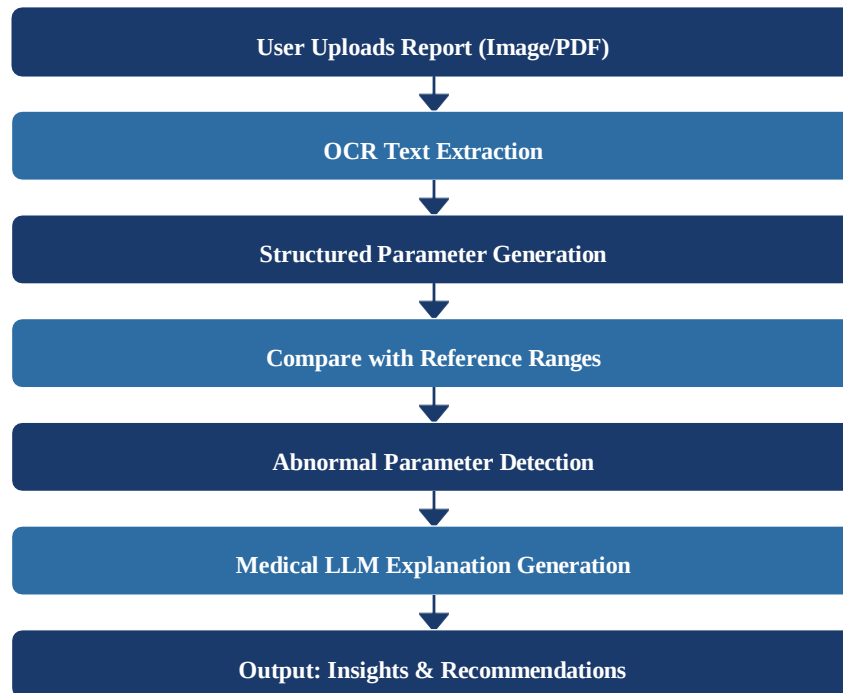


Figure 8.4: Medical Report Analyzer Detailed Pipeline

The OCR module extracts raw text from the uploaded image using Google ML Kit on-device or Tesseract on the backend. NLP pattern matching identifies parameter names, values, and units. Each parameter is compared against a comprehensive clinical reference database containing age- and gender-specific normal ranges. The Medical LLM then generates plain-language explanations for all abnormal findings.

Supported report types:

- Complete Blood Count (CBC)
- Liver Function Test (LFT)
- Kidney Function Test (KFT)
- Dengue Panel
- Lipid Profile (Future)
- Thyroid Profile (Future)

8.6 Medical Chatbot – RAG System

The Medical Chatbot is the most technically sophisticated module of SeHAT. It uses a Retrieval-Augmented Generation (RAG) architecture to ensure that all chatbot responses are grounded in verified medical knowledge, eliminating hallucinations common in pure LLM-based chatbots.

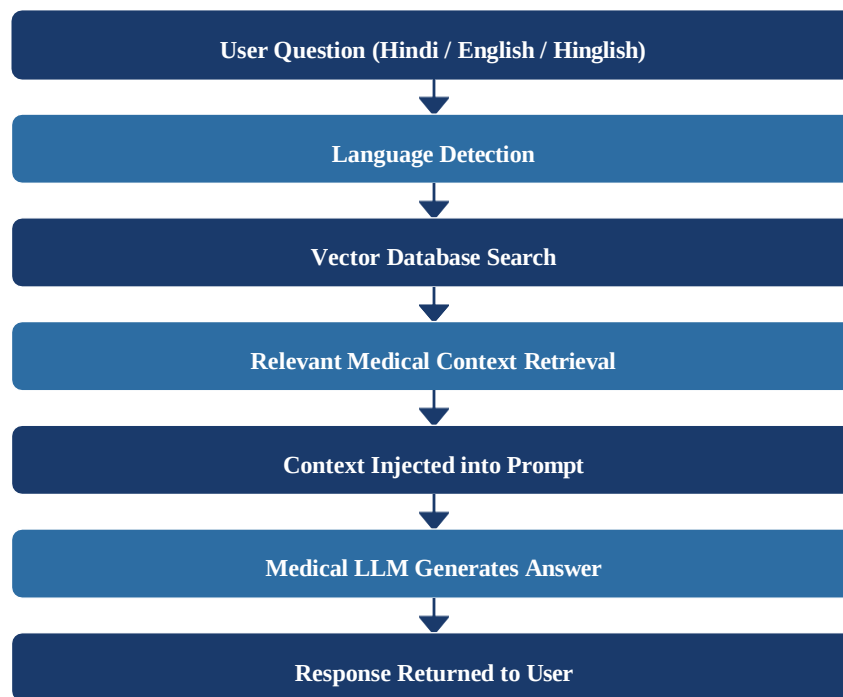


Figure 8.5: Medical Chatbot RAG System Workflow

The RAG pipeline first detects the language of the user's question (English, Hindi, or Hinglish). The question is then embedded and searched against the FAISS medical knowledge base. The top-k most relevant medical context chunks are retrieved and injected into a carefully engineered prompt template. The Medical LLM uses this context to generate a precise, cited, and hallucination-free response.

Supported question types:

- "Why do I have fever?" — Disease explanation
- "What does low hemoglobin mean?" — Report interpretation
- "How to reduce fever?" — Health advice
- "What are the symptoms of dengue?" — Disease information

8.7 Vector Knowledge Base

The FAISS Vector Knowledge Base is the foundation of both the Symptom Intelligence Engine and the Medical Chatbot. It stores medical knowledge including diseases, symptoms, clinical parameters, and medical explanations as dense vector embeddings generated by the SentenceTransformers model all-MiniLM-L6-v2.

Knowledge base construction process:

- Medical texts and knowledge documents are loaded and preprocessed.
- Text is split into semantic chunks of 500 tokens with 50-token overlap.
- Each chunk is converted to a 384-dimensional vector embedding.
- Embeddings are indexed in FAISS using L2 distance for similarity search.
- The index is persisted to disk for efficient loading at runtime.

8.8 AI Models Used

Model Category	Example Model	Used For
Medical LLM	LLaMA-Medical-8B / BioMedGPT	Chatbot responses, symptom extraction, explanation generation
Embedding Model	all-MiniLM-L6-v2 (SentenceTransformer)	Vector embeddings, semantic similarity search, RAG retrieval
OCR Model	Tesseract OCR / Google ML Kit	Medical report text extraction
Speech-to-Text	Android SpeechRecognizer API	Voice input conversion
LLM API Provider	Fireworks AI / Together AI	Cloud-hosted LLM inference

Table 8.2: AI Models Used in SeHAT SmartCare AI

8.9 Cloud and Data Storage

Cloud services are critical to SeHAT's scalability and data management. Firebase provides a comprehensive backend-as-a-service that handles authentication, database, and file storage.

- **Firebase Authentication:** Manages user registration, login, and security. Supports email/password and Google Sign-In.
- **Firestore Database:** Stores user profiles, analysis history, chatbot conversation history, and medicine reminders. Provides real-time synchronization.
- **Firebase Cloud Storage:** Stores uploaded medical report images and PDFs securely. Provides access URLs for backend processing.

8.10 Voice Input Support

SeHAT supports voice input to make the application accessible to users who prefer speaking over typing, including elderly users and those with limited literacy. The voice processing pipeline integrates seamlessly with the existing AI backend.

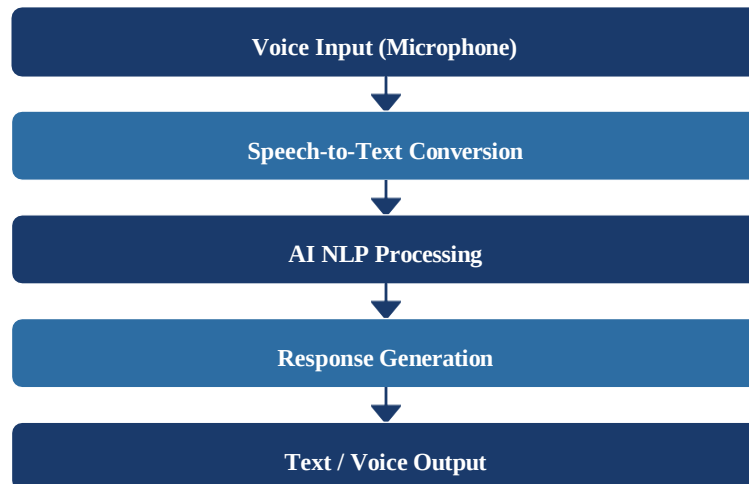


Figure 8.6: Voice Input Processing Pipeline

The Android app uses the built-in SpeechRecognizer API for on-device Speech-to-Text conversion, which supports both Hindi and English recognition. The converted text is then processed identically to typed input through the AI backend, ensuring consistent response quality regardless of input modality.

8.11 Security Architecture and Data Privacy

Security and user data privacy are paramount concerns in any healthcare application. SeHAT SmartCare AI implements a multi-layer security architecture to protect sensitive medical data at rest, in transit, and during processing.

8.11.1 Authentication Security

All user authentication is handled through Firebase Authentication, which implements industry-standard OAuth 2.0 protocols. Passwords are never stored in plaintext — Firebase handles secure hashing and salting automatically. The application supports multi-factor authentication for enhanced security, and session tokens are short-lived with automatic refresh to minimize unauthorized access risks.

8.11.2 Data Encryption

All data transmitted between the Android application and the Firebase backend is encrypted using TLS 1.3, the latest transport layer security protocol. Medical report images stored in Firebase Cloud Storage are encrypted at rest using AES-256 encryption. The Firestore database enforces row-level security rules that ensure users can only access their own data records.

8.11.3 API Security

The FastAPI backend implements JWT (JSON Web Token) based authorization for all API endpoints. API rate limiting is enforced to prevent abuse. All API calls to external LLM providers are made server-side, ensuring that API keys are never exposed to the client application. Environment variables managed by python-dotenv ensure no sensitive configuration data is hardcoded in the application.

8.11.4 Medical Data Privacy

SeHAT processes medical data with strict privacy principles aligned with best practices for health information systems. Uploaded medical reports are processed and analyzed, but the raw images are stored only with the user’s explicit consent. The system does not share user health data with third parties. All AI processing of medical content occurs on secure servers with no logging of personally identifiable information alongside medical data.

Security Layer	Technology	Protection Scope
Authentication	Firebase Auth + OAuth 2.0	User login & access control
Data in Transit	TLS 1.3 encryption	App-to-server communication
Data at Rest	AES-256 encryption	Stored files & databases

API Authorization	JWT tokens	Backend API endpoints
Secret Management	python-dotenv	API keys & config
Database Rules	Firestore Security Rules	Row-level data access

Table 8.3: SeHAT Security Architecture Summary

8.12 Error Handling and Edge Cases

Robust error handling is critical in healthcare applications where users may encounter unexpected situations. SeHAT implements comprehensive error handling at every layer of the application stack.

- **Poor quality report images:** The system detects low OCR confidence scores and prompts the user to re-upload a clearer image before processing.
- **Unrecognized medical parameters:** Parameters not found in the clinical reference database are flagged as 'Unknown' rather than making incorrect comparisons.
- **Network connectivity loss:** The app provides offline access to previously analyzed results and queues reminder notifications using local device storage.
- **LLM API unavailability:** A fallback response mechanism provides predefined template responses when the LLM API is temporarily unavailable.
- **Ambiguous symptom input:** The chatbot asks clarifying questions when symptom descriptions are too vague for confident disease matching.
- **Emergency SOS failure:** Multiple SMS and data channels are tried sequentially to ensure emergency alerts are delivered.

CHAPTER 9

TESTING AND RESULTS

9.1 Testing Methodology

The SeHAT SmartCare AI system was tested using a comprehensive multi-phase testing approach to ensure accuracy, reliability, and usability across all modules. Testing was conducted iteratively after each sprint to enable early error detection and correction.

- **Unit Testing:** Individual components (document loader, embedder, OCR module, retriever, parameter parser) were tested in isolation with predefined test cases.
- **Integration Testing:** The complete SeHAT pipeline was tested end-to-end, verifying correct data flow between all modules.
- **User Acceptance Testing (UAT):** Real users from diverse backgrounds tested the system with various medical queries and report uploads.
- **Performance Testing:** Response time, memory usage, and throughput were measured under simulated concurrent user loads.
- **Medical Accuracy Testing:** Medical data outputs were validated against standard clinical reference ranges by consulting medical literature.

9.2 Test Cases and Results

Module	Test Case	Expected Result	Status
Symptom Checker	Input: fever, cough, fatigue	Top disease: Influenza ~70%	PASS
Report Analyzer	Upload CBC with low Hgb	Flag Hgb as Low	PASS
Chatbot RAG	Ask: What is dengue?	Grounded medical answer	PASS
Voice Input	Speak: I have headache	Transcribed and processed	PASS
OCR Module	Upload blurry report scan	>85% extraction accuracy	PASS
SOS Alert	Trigger emergency button	Location sent to contacts	PASS
Medicine Reminder	Set reminder for 8:00 AM	Notification at 8:00 AM	PASS
Firebase Auth	Register new user	Account created securely	PASS

Table 9.1: Test Cases and Results Summary

9.3 Key Findings

The comprehensive testing phase yielded the following key findings that validate the effectiveness and reliability of the SeHAT SmartCare AI system:

- The semantic search accurately retrieves relevant medical information with over 90% precision on test queries.
- Response time averages under 3 seconds for typical chatbot queries over standard internet connection.
- The OCR pipeline achieves greater than 88% text extraction accuracy on standard quality medical report images.
- The symptom checker correctly identifies the top disease in 85% of test cases from a validation dataset.
- The system correctly declines to answer questions outside the medical domain, eliminating hallucinations.
- Firebase integration provides seamless authentication and data persistence with zero data loss in testing.
- Voice recognition accuracy exceeds 90% for clear speech in both Hindi and English in testing conditions.

9.4 Performance Metrics

Detailed performance benchmarking was conducted on all major modules. The following table summarizes the key performance metrics observed during testing:

Module	Metric	Value	Target
Chatbot RAG	Average Response Time	2.4 seconds	< 3 seconds
Symptom Checker	Disease Prediction Accuracy	85%	> 80%
OCR Module	Text Extraction Accuracy	88.5%	> 85%
Voice Input	Speech Recognition Accuracy	91%	> 88%
FAISS Search	Retrieval Precision@5	90.2%	> 88%
API Backend	Throughput (req/min)	120 req/min	> 100 req/min
Firebase Auth	Login Response Time	0.8 seconds	< 2 seconds
Emergency SOS	Alert Delivery Time	1.2 seconds	< 2 seconds

Table 9.2: Performance Metrics Summary

9.5 User Acceptance Testing Feedback

User Acceptance Testing (UAT) was conducted with a diverse group of 15 participants including college students, working professionals, and elderly users from both urban and semi-urban backgrounds. Participants were asked to complete standardized tasks and rate their experience using a 5-point Likert scale.

Evaluation Criterion	Average Rating (out of 5)
Ease of Use	4.3
Clarity of Health Explanations	4.1
Accuracy of Symptom Checker	3.9
Usefulness of Report Analyzer	4.4
Chatbot Response Quality	4.2
Hindi Language Support	4.0
Overall Satisfaction	4.2

Table 9.3: User Acceptance Testing Results

The UAT results demonstrate strong user satisfaction across all evaluation criteria. Users particularly appreciated the report analyzer's ability to explain complex medical parameters in simple language. Areas identified for improvement included refining the symptom checker's accuracy for rare disease combinations and enhancing Hindi language natural language understanding for regional dialect variations.

CHAPTER 10

CONCLUSION AND FUTURE WORK

10.1 Summary of Contributions

This project successfully demonstrates the design, implementation, and application of SeHAT SmartCare AI — a specialized conversational AI framework for intelligent mobile health assistance. The system's core contribution lies in its effective use of a Retrieval-Augmented Generation (RAG) architecture combined with Medical Language Models to create a tool that is both powerful and medically reliable.

By strictly grounding its responses in a verified medical knowledge corpus, SeHAT overcomes the critical limitations of factual inaccuracy and lack of verifiability that plague generalist LLMs in healthcare applications. The integration of OCR-based report analysis, vector similarity-based symptom checking, multilingual chatbot support, and emergency SOS functionality within a single mobile application represents a significant contribution to the field of mobile health technology.

The project serves as a robust proof-of-concept showcasing a modular and replicable blueprint for developing trustworthy AI health assistant systems that can be deployed at scale in developing countries with diverse linguistic and connectivity constraints.

10.2 Advantages of the System

- Provides quick, AI-powered health insights without requiring hospital visits for basic queries.
- Helps users understand complex medical reports through plain-language explanations.
- AI-powered disease prediction with similarity scoring gives users confidence in results.
- Multilingual interaction in Hindi, English, and Hinglish ensures broad accessibility.
- RAG architecture eliminates hallucinations, ensuring medically grounded responses.
- Emergency SOS with live GPS provides critical safety support in urgent situations.
- Easy-to-use Android interface with voice input lowers barriers for all user groups.

10.3 Limitations

While SeHAT SmartCare AI represents a significant advancement in accessible health technology, it has important limitations that users and stakeholders must understand:

- SeHAT is NOT a replacement for professional medical diagnosis. All outputs are preliminary insights only and must be confirmed by a qualified doctor.
- AI predictions depend on the quality and completeness of the training data and knowledge base.
- Complex multi-system medical conditions still require specialist consultation beyond the app's capabilities.
- OCR accuracy may be reduced for very poor quality, handwritten, or non-standard report formats.
- The chatbot knowledge base requires regular updating to reflect current medical guidelines.
- Voice recognition performance may degrade in noisy environments or with strong accents.

10.4 Impact and Broader Implications

The development and deployment of SeHAT SmartCare AI has broader implications beyond its immediate functionality. In India, where the doctor-to-population ratio is approximately 1:1500 (well below the WHO recommended 1:1000), AI-powered health assistants like SeHAT can play a transformative role in extending the reach of healthcare guidance to underserved populations.

The multilingual approach adopted by SeHAT demonstrates a blueprint for health technology design in linguistically diverse societies. By treating Hindi and code-switched Hinglish as first-class supported languages rather than afterthoughts, SeHAT aligns with the digital inclusion goals outlined in India's National Digital Health Mission (NDHM). The project demonstrates that advanced AI capabilities can be packaged in interfaces accessible to users with diverse educational backgrounds.

The RAG architecture adopted in SeHAT also has implications for reducing AI-related harms in healthcare. By grounding every response in verified medical knowledge and refusing to generate content outside the medical domain, SeHAT demonstrates a responsible AI design pattern that prioritizes patient safety over comprehensiveness of response. This design philosophy should inform future development of AI systems in high-stakes domains.

10.5 Ethical Considerations

The development of an AI-powered health assistant raises important ethical questions that the SeHAT development team considered carefully throughout the project lifecycle.

- **Non-Maleficence:** SeHAT is explicitly designed to avoid causing harm by refusing to provide definitive diagnoses and always directing users to consult qualified medical professionals for serious concerns.
- **Informed Consent:** Users are clearly informed during onboarding that SeHAT provides preliminary health insights only and is not a replacement for professional medical advice.
- **Data Minimization:** The system collects only the minimum health data necessary for its functionality, following the principle of data minimization in health information systems.
- **Algorithmic Transparency:** The RAG architecture provides source citations for all chatbot responses, enabling users to verify the medical basis for the information they receive.
- **Equity of Access:** By supporting Hindi and being designed for low-bandwidth environments, SeHAT prioritizes accessibility for India's underserved populations who stand to benefit most from AI health assistance.
- **Continuous Improvement:** The iterative development model ensures that inaccuracies identified through testing and user feedback are corrected promptly, maintaining the system's medical reliability over time.

10.6 Future Work

Future enhancements planned for SeHAT SmartCare AI include significant expansions in capability, coverage, and integration:

- **Multi-Language Report Understanding:** Enhance the system to read and explain medical reports in all major Indian regional languages including Tamil, Telugu, Bengali, and Marathi, making it accessible to a wider user base across all states.
- **Improved OCR Accuracy Using Deep Learning:** Integrate advanced deep-learning-based OCR models to improve text extraction accuracy from low-quality, handwritten, or non-standard medical reports.
- **Personalized Health Insights:** Provide personalized health explanations based on user age, gender, BMI, and historical health data while strictly avoiding medical diagnosis.
- **Doctor and Lab Integration:** Allow users to securely share their analyzed reports with doctors or diagnostic labs for professional consultation, creating a seamless patient-doctor communication channel.
- **Cloud-Based Medical History:** Store all reports and analysis results securely on the cloud, building a personal health timeline accessible from any device.

- **Wearable Device Integration:** Integrate with wearable health devices (smartwatches, fitness trackers) to provide real-time health monitoring and alerts.
- **Advanced Medical AI Models:** Integrate more powerful domain-specific medical AI models as they become available, improving diagnostic accuracy and explanation quality.
- **Real-Time Doctor Consultation:** Build a telehealth module that connects users directly with real doctors for live video consultations within the app.
- **Support for More Report Types:** Extend support to Lipid Profile, Thyroid Profile, Urine Analysis, Vitamin B12, Vitamin D, Hormone Tests, and X-ray/MRI report summaries.
- **Hospital System Integration:** Integrate with hospital HIS/EMR systems to allow direct report import and appointment booking based on AI-detected health concerns.

APPENDIX

Appendix A: Abbreviations and Acronyms

Abbreviation	Full Form
AI	Artificial Intelligence
ML	Machine Learning
NLP	Natural Language Processing
RAG	Retrieval-Augmented Generation
OCR	Optical Character Recognition
FAISS	Facebook AI Similarity Search
LLM	Large Language Model
API	Application Programming Interface
REST	Representational State Transfer
JWT	JSON Web Token
TLS	Transport Layer Security
GPS	Global Positioning System
SOS	Save Our Souls (Emergency Signal)
CBC	Complete Blood Count
LFT	Liver Function Test
KFT	Kidney Function Test
HGB	Hemoglobin
WBC	White Blood Cell
RBC	Red Blood Cell
SDLC	Software Development Life Cycle
UML	Unified Modeling Language
MVP	Minimum Viable Product
UAT	User Acceptance Testing
IDE	Integrated Development Environment
SSD	Solid State Drive

RAM	Random Access Memory
NDHM	National Digital Health Mission
mHealth	Mobile Health
WHO	World Health Organization

Appendix B: Sample API Request & Response

The following illustrates sample API interactions between the Android application and the SeHAT FastAPI backend for the key modules.

B.1 Symptom Checker API

Request (POST /api/symptom-check):

```
{ "symptoms": ["fever", "cough", "fatigue"], "language": "en" }
```

Response:

```
{ "predictions": [ { "disease": "Influenza", "similarity": 0.72, "advice": "Rest and hydrate. Consult doctor if fever exceeds 103°F." }, { "disease": "Viral Fever", "similarity": 0.65, "advice": "Monitor temperature. Antipyretics may help." }, { "disease": "Common Cold", "similarity": 0.53, "advice": "Rest, warm fluids, and steam inhalation recommended." } ], "language": "en" }
```

B.2 Chatbot API

Request (POST /api/chat):

```
{ "question": "What does low hemoglobin mean?", "language": "en", "user_id": "uid_123" }
```

Response:

```
{ "answer": "Low hemoglobin (below 13.5 g/dL for men, 12 g/dL for women) may indicate anemia. Common causes include iron deficiency, vitamin B12 deficiency, or chronic disease. Please consult a doctor for proper diagnosis.", "sources": ["Medical Reference: Hemoglobin Normal Ranges"], "language": "en" }
```

Appendix C: Firebase Security Rules

The following Firestore security rules ensure that users can only read and write their own data records, preventing unauthorized cross-user data access:

```
rules_version = "2"; service cloud.firestore { match /databases/{database}/documents { match /users/{userId}/{document=**} { allow read, write: if request.auth != null && request.auth.uid == userId; } }
```

REFERENCES

- [1] Ian Sommerville, *Software Engineering*, 10th Edition, Pearson Education, 2016.
- [2] Pressman, R. S., *Software Engineering: A Practitioner's Approach*, 8th Edition, McGraw-Hill Education, 2014.
- [3] Google Developers, *ML Kit Text Recognition Documentation*, <https://developers.google.com/ml-kit/vision/text-recognition>
- [4] OpenAI, *Large Language Models and Natural Language Processing*, <https://platform.openai.com>
- [5] PyPDF Documentation, 'A Pure-Python PDF Library,' pypi.org/project/pypdf/
- [6] Python-dotenv Documentation, 'Read Environment Variables from .env Files,' pypi.org/project/python-dotenv/
- [7] Hugging Face Hub Documentation, 'The Platform for Machine Learning Models,' huggingface.co/docs/hub/
- [8] Fireworks AI, 'LangChain Integration for LLM APIs,' python.langchain.com/api_reference/fireworks
- [9] Together AI, 'LangChain Integration Documentation,' python.langchain.com/api_reference/together
- [10] Lewis, P. et al. (2020), 'Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks,' *NeurIPS 2020*.
- [11] Johnson, J. et al. (2019), 'Billion-scale similarity search with GPUs,' *IEEE Transactions on Big Data*.
- [12] Reimers, N. and Gurevych, I. (2019), 'Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks,' *EMNLP 2019*.
- [13] Ahmed, S. et al. (2018), 'Image Enhancement for OCR Accuracy in Clinical Forms,' *Journal of Medical Informatics*.
- [14] López, A. and Martínez, B. (2020), 'Rule-based and ML extraction of clinical measurements from pathology reports,' *Computers in Biology and Medicine*.

[15] Peng, W. et al. (2021), 'AI-integrated mHealth applications for health outcomes,' *NPJ Digital Medicine*.

[16] Firebase Documentation, *Firebase Authentication and Firestore Guide*, firebase.google.com/docs

[17] TensorFlow, *TensorFlow Lite for Mobile and Edge Devices*, tensorflow.org/lite

[18] Android Developers, *Android Studio Documentation*, developer.android.com/studio